

1ª Questão: FEITA

1. [1.0] Cite três desvantagens da técnica de implementação mono-ciclo, justifique suas respostas.

resposta: A técnica de mono-ciclo pode ser considerada menos vantajosa pelo fato de ter um baixo **throughput**, uma vez que uma única instrução é executada em um ciclo de clock. Não pode ter pipeline.

Duplicação de componentes devido a restrição de um ciclo para o processamento de instrução

Ciclo de clock longo para comportar instrução mais lenta

Instruções que demandam menos tempo deixam CPU ociosa

Implementação pouco eficiente

2ª Questão: Considere o programa em linguagem assembly do LEGv8 descrito abaixo. Os registradores X2 tem o endereço do array.

```
loop:    ADDI X3, XZR, #64
         LDUR X5, [X2, #0]
         LDUR X6, [X2, #4]
         ADD X5, X5, X6
         STUR X5, [X2, #0]
         ADD X6, X6, X6
         ADD X6, X6, X1
         STUR X6, [X2, #-8]
         ADDI X2, X2, #8
         SUBI X3, X3, #1
         CBNZ X3, loop
```

Considere uma implementação em pipeline de 5 estágios conforme apresentada no livro texto. Os conflitos de dados e controle devem ser resolvidos com a inserção de NOPs. Calcule o CPI do programa para a implementação em pipeline. Justifique sua resposta.

Ciclos/numero de instruções

25/11

1 iteração tem 20 linhas -> 64 iterações 1280 linhas

+1 da primeira instrução

+4 pra fechar o clock

1285 ciclos de clock

1 iteração tem 10 instruções -> 64 iterações 640 instruções

+1 da primeira instrução
641 instruções
 $CPI = 1285/641$

CÓDIGO:

```
ADDI X3, XZR, #64
LDUR X5, [X2, #0]
LDUR X6, [X2, #8]
NOP
NOP
ADD X5, X5, X6
NOP
NOP
STUR X5, [X2, #0]
ADD X6, X6, X6
NOP
NOP
ADD X6, X6, X1
NOP
NOP
STUR X6, [X2, #-8]
ADDI X2, X2, #8
SUBI X3, X3, #1
NOP
NOP
CBNZ X3, loop
```

3ª Questão: Considere agora uma implementação do pipeline com os conflitos de dados resolvidos com unidade de adiantamento e unidade de detecção de conflito. O desvio passou a ser resolvido no estágio ID. Calcule o novo CPI e o speed-up obtido em relação à questão anterior. Justifique sua resposta.

resposta:

1 iteração tem 11 linhas -> 64 iterações 704 linhas
+1 da primeira instrução
+4 pra fechar o clock
709 ciclos de clock
1 iteração tem 10 instruções -> 64 iterações 640 instruções
+1 da primeira instrução
641 instruções
 $CPI = 709/641$

CÓDIGO:

```
ADDI X3, XZR, #64
LDUR X5, [X2, #0]
LDUR X6, [X2, #8]
NOP
ADD X5, X5, X6
STUR X5, [X2, #0]
ADD X6, X6, X6
ADD X6, X6, X1
STUR X6, [X2, #-8]
ADDI X2, X2, #8
SUBI X3, X3, #1
CBNZ X3, loop
```

4ª Questão: Explique como funciona um preditor de 2 bits, qual o novo CPI se o pipeline tiver previsão de desvio (preditor de 2 bits) com estado inicial do preditor igual a (00)? Justifique sua resposta.

resposta:

preditor 2 bits: O preditor de 2-bits (bimodal) é essencialmente um contador de dois bits com valores entre 0 e 3 • Quando o contador é maior ou igual ao valor 2, o desvio é predito como tomado; em caso contrário é predito como não tomado • O contador é incrementado em um desvio tomado e decrementado em um desvio não tomado

$$712 = (61 \cdot 11) + (3 \cdot 12) + 4 + 1$$

61 instruções que não tem erro, então tem 11 linhas

3 instruções que tem erro, então tem 12 linhas

+4 pra fechar o clock

+1 pra adicionar a primeira instrução que ta fora do loop

$$641 = 10 \cdot 64 + 1$$

64 loops com 10 instruções cada

+1 pra adicionar a primeira instrução que ta fora do loop

$$\text{CPI} = 712 \text{ clocks} / 641 \text{ instruções}$$

5ª Questão: Considere agora que o pipeline da questão 3 foi implementado como um superescalar com two-issue estático, que permite executar instruções aritméticas/shift e loads em paralelo, bem como desvio e store.

- Rearrume o código e calcule o novo CPI para o programa da questão 2 para o processador com despacho múltiplo two-issue estático. Compare com o CPI da questão 3, justificando os valores obtidos.

resposta Comparação: $13 (\text{CPI two-issue}) / \text{CPI questão 3}$

1 iteração tem 8 linhas -> 64 iterações tem 512

+4 pra fechar o clock

+1 da última instrução

não soma 1 porque ela ta junto com a iteração

517 ciclos de clock

449 instruções

CPI = 517/449

ADDI X3, XZR, #64
loop:
LDUR X5, [X2, #0]
LDUR X6, [X2, #8]
NOP
ADD X5, X5, X6
STUR X5, [X2, #0]
ADD X6, X6, X6
ADD X6, X6, X1
STUR X6, [X2, #-8]
ADDI X2, X2, #8
SUBI X3, X3, #1
CBNZ X3, loop
LDUR X5, [X2, #0]
LDUR X6, [X2, #8]
NOP
ADD X5, X5, X6
STUR X5, [X2, #0]
ADD X6, X6, X6
ADD X6, X6, X1
STUR X6, [X2, #-8]
ADDI X2, X2, #8
SUBI X3, X3, #1
CBNZ X3, loop

- b. Use a técnica de desenrolar loop no programa, qual o novo CPI obtido?
Justifique sua resposta.

resposta

Não teve alteração no loop/cpi (ficará igual a anterior se considerar two-issue com suporte para adiantamento)

```
ADDI X3, XZR, #64
LDUR X5, [X2, #0]
LDUR X6, [X2, #8]
ADD X5, X5, X6
STUR X5, [X2, #0]
ADD X6, X6, X6
ADD X6, X6, X1
STUR X6, [X2, #-8]
ADDI X2, X2, #8
SUBI X3, X3, #1
CBNZ X3, loop
FOI A MESMA COITA?
CPI = 1,15
```

6ª Questão: Explique os conceitos de superescalar e execução fora de ordem usando menos que 20 palavras para cada explicação. Estes conceitos são ortogonais? Sim ou não? Justifique sua resposta.

Superescalar:

Uma implementação do pipeline que permite que o processador execute mais de uma instrução por ciclo de clock selecionando elas durante a execução.

Execução fora de ordem:

Situação quando uma instrução que está com a execução bloqueada não faz com que as próximas instruções precisem esperar.

Não são ortogonais, é possível existir processadores superescalares que realizam execuções fora de ordem.

resposta:

superescalar1: Na arquitetura superescalar, várias instruções podem ser iniciadas simultaneamente e executadas independentemente umas das outras.

superescalar2: Um superescalar CPU pode executar mais de uma instrução por ciclo de clock. Como as velocidades de processamento são medidas em ciclos de clock por segundo (Megahertz), um processador superescalar será mais rápido que um processador escalar classificado no mesmo megahertz.

Uma arquitetura superescalar inclui unidades de execução paralela, que podem executar instruções simultaneamente. Essa arquitetura paralela foi implementada pela primeira vez em RISC processadores, que usam instruções curtas e simples para realizar cálculos. Devido às suas capacidades superescalares, os processadores RISC normalmente têm um desempenho melhor do que CISC processadores rodando no mesmo megahertz. No entanto, a maioria dos processadores baseados em CISC (como o Intel Pentium) agora também inclui alguma arquitetura RISC, o que lhes permite executar instruções em paralelo. Quase todos os processadores desenvolvidos após o 1998 são superescalares.

não sei se é ortogonal, mas ortogonal é isso aqui: Dentro engenharia da computação, a conjunto de instruções ortogonais é um arquitetura do conjunto de instruções onde todos os tipos de instrução podem usar todos modos de endereçamento. Isto é "ortogonal"no sentido de que o tipo de instrução e o modo de endereçamento variam independentemente. Um conjunto de instruções ortogonais não impõe uma limitação que requer uma certa instrução para usar um registro específico[1] portanto, há pouca sobreposição de funcionalidade de instrução.[2]

execução fora de ordem: A Execução Fora-de-ordem permite que uma instrução pronta para ser processada seja executada antes mas não tem os operandos prontos, em seguida, re-ordens os resultados para fazer parecer que tudo aconteceu na ordem programada. Esta técnica também é usada para evitar outras “bolhas”, como uma instrução aguardando um resultado de uma operação de ponto flutuante de longa latência ou outras operações multi-ciclo.

7ª Questão: Explique como a anti-dependência e a dependência de saída são resolvidas nas técnicas de especulação dinâmica, e como é garantido que as previsões erradas não introduzem erros no programa.
resposta

https://edisciplinas.usp.br/pluginfile.php/4109781/mod_resource/content/5/5aula%20-%20Pipeline.pdf#:~:text=Antidepend%C3%Aancia%20ou%20WAR%20

A anti-dependência (write-after-read) é resolvida executando os estágios da instrução(anterior), que está utilizando o registrador como leitura, antes da instrução que está utilizando o mesmo registrador como escrita. A dependência de saída (write-after-write) é resolvida executando os estágios da instrução(anterior), que está utilizando o registrador como escrita, antes da instrução que está utilizando o mesmo registrador como escrita. As previsões erradas garantem a correta execução do programa, pois quando ocorre, acontece um flush em todas as instruções que foram computadas no momento da previsão errada.

Execução Especulativa dinâmica com preditor de 2 bits é a técnica que permite o processador executar instruções antecipadamente, especulando que operações serão necessárias nos próximos ciclos com base nas informações coletadas em tempo de execução (considerando os 2 bits na BHT - *Branch History Table*. Primeiro bit indica o resultado da previsão, ou seja, o que provavelmente irá acontecer, e o segundo bit o que aconteceu na última execução - bit 1 se desviou e bit 0 se não desviou - fornecendo uma segunda oportunidade de acerto). **Vantagens:** Como não precisa esperar a resolução do branch para carregar e executar as instruções seguintes, podemos obter um cpi melhor. **Desvantagens:** Há a necessidade de usar um hardware específico com a presença de um *reorder buffer* e do próprio preditor de 2 bits. Além disso, também existiria o problema de erro na especulação, sendo necessário o flush de instruções que haviam sido/estavam sendo executadas.

Anti-dependência: resolvemos executando primeiro os estágios de uma instrução anterior que está usando um determinado registrador em modo leitura e só depois executamos os estágios da instrução que usa o mesmo registrador em modo de escrita. **Dependência de saída:** É resolvida de forma parecida, a diferença é que a instrução anterior estará usando o registrador também como modo de escrita, ou seja, os dois estão usando o registrador como modo escrita.

